



ITESO

**Universidad Jesuita
de Guadalajara**

Jorge Sebastian Sanchez Arana

Desarrollo de aplicaciones y Servicios Web

Miguel Alejandro Villa Guerrero

Jueves 20 de Marzo 2025

Introducción

En esta práctica se implementó un sistema para la gestión de productos y un carrito de compras utilizando JavaScript. El sistema permite crear, actualizar, eliminar y buscar productos, así como gestionar un carrito de compras donde se pueden agregar, actualizar, eliminar productos y calcular el total de la compra. Este tipo de sistemas es fundamental en plataformas de e-commerce y gestión de inventarios.

Objetivo de la práctica

El objetivo de esta práctica fue implementar un sistema de gestión de productos y un carrito de compras, permitiendo realizar las siguientes operaciones:

Gestión de productos:

- Crear productos con diversas propiedades como título, descripción, precio, etc.
- Actualizar productos existentes.
- Eliminar productos.
- Buscar productos por categoría y título.

Carrito de compras:

- Agregar productos al carrito.
- Actualizar la cantidad de productos en el carrito.
- Eliminar productos del carrito.
- Calcular el total del carrito de compras.

Desarrollo

El sistema se compone de varios módulos que interactúan entre sí. A continuación, se describen los módulos principales:

utils.js:

Contiene la función `generateUUID()`, la cual genera un identificador único (UUID) para los productos. Esto asegura que cada producto tenga un identificador único.

```
1
2 export function generateUUID() {
3   return "xxxxxxxx-xxxx-4xxx-yxxx-xxxxxxxxxxxx".replace(/[xy]/g, (c) => {
4     let r = (Math.random() * 16) | 0;
5     let v = c == "x" ? r : (r & 0x3) | 0x8;
6     return v.toString(16);
7   });
8 }
```

products.js:

Define la clase `Product`, que representa un producto con propiedades como título, descripción, imagen, precio, stock y categoría. Esta clase también incluye métodos de validación para asegurar que los datos sean correctos.

```
1 export class ProductException extends Error {
2   constructor(message) {
3     this.message = message;
4     this.name = "ProductException";
5   }
6 }
```



```
1  export class Product {
2    constructor(
3      title,
4      description,
5      imageUrl,
6      unit,
7      stock,
8      pricePerUnit,
9      category
10   ) {
11     this._uuid = generateUUID();
12     this.setTitle(title);
13     this.setDescription(description);
14     this.setImageUrl(imageUrl);
15     this.setUnit(unit);
16     this.setStock(stock);
17     this.setPricePerUnit(pricePerUnit);
18     this.setCategory(category);
19   }
20 }
```



```
1  get uuid() {  
2      return this._uuid;  
3  }  
4  
5  get title() {  
6      return this._title;  
7  }  
8  
9  get description() {  
10     return this._description;  
11 }  
12  
13 get imageUrl() {  
14     return this._imageUrl;  
15 }  
16  
17 get unit() {  
18     return this._unit;  
19 }  
20  
21 get stock() {  
22     return this._stock;  
23 }  
24  
25 get pricePerUnit() {  
26     return this._pricePerUnit;  
27 }  
28  
29 get category() {  
30     return this._category;  
31 }
```

```
1 setTitle(title) {
2     if (!title || title.trim() === "") {
3         throw new ProductException("Title error (String error)");
4     }
5     this._title = title;
6 }
7
8 setDescription(description) {
9     if (!description || description.trim() === "") {
10        throw new ProductException("Description error (String error)");
11    }
12    this._description = description;
13 }
14
15 setImageUrl(imageUrl) {
16     if (!imageUrl || imageUrl.trim() === "") {
17         throw new ProductException("ImageURL error (String error)");
18     }
19     this._imageUrl = imageUrl;
20 }
21
22 setUnit(unit) {
23     if (!unit || unit.trim() === "") {
24         throw new ProductException("Unit error (String error)");
25     }
26     this._unit = unit;
27 }
28
29 setStock(stock) {
30     if (stock < 0) {
31         throw new ProductException("Stock error (Number error)");
32     }
33     this._stock = stock;
34 }
35
36 setPricePerUnit(pricePerUnit) {
37     if (pricePerUnit < 0) {
38         throw new ProductException("Price error (Number error)");
39     }
40     this._pricePerUnit = pricePerUnit;
41 }
42
43 setCategory(category) {
44     if (!category || category.trim() === "") {
45         throw new ProductException("Category error (String error)");
46     }
47     this._category = category;
48 }
```

```

1  toHTML() {
2      return `
3          <div class="product">
4              
5              <h3>${this.title}</h3>
6              <p>${this.description}</p>
7              <p><strong>Price:</strong> ${this.pricePerUnit}</p>
8              <p><strong>Category:</strong> ${this.category}</p>
9              <p><strong>Stock:</strong> ${this.stock} units</p>
10         </div>
11     `;
12 }
13
14 static createFromJson(jsonValue) {
15     const obj = JSON.parse(jsonValue);
16     return new Product(
17         obj.title,
18         obj.description,
19         obj.imageUrl,
20         obj.unit,
21         obj.stock,
22         obj.pricePerUnit,
23         obj.category
24     );
25 }
26
27 static createFromObject(obj) {
28     return new Product(
29         obj.title,
30         obj.description,
31         obj.imageUrl,
32         obj.unit,
33         obj.stock,
34         obj.pricePerUnit,
35         obj.category
36     );
37 }
38
39 static cleanObject(obj) {
40     const validKeys = [
41         "title",
42         "description",
43         "imageUrl",
44         "unit",
45         "stock",
46         "pricePerUnit",
47         "category",
48     ];
49     const cleanObj = {};
50     for (const key in obj) {
51         if (validKeys.includes(key)) {
52             cleanObj[key] = obj[key];
53         }
54     }
55     return cleanObj;
56 }
57 }

```

data_handler.js:

Gestiona un array de productos (**products**), y proporciona funciones para agregar, actualizar, eliminar y buscar productos. Además, incluye la función **productListToHTML** que convierte la lista de productos en código HTML.

```
1  import { Product, ProductException } from './products.js';
2
3  let products = [];
4
5  export function getProducts() {
6    return products;
7  }
8
9  export function getProductById(uuid) {
10   return products.find((product) => product.uuid === uuid);
11 }
12
13 export function createProduct(product) {
14   if (!(product instanceof Product)) {
15     throw new Error("Producto Invalido");
16   }
17   products.push(product);
18 }
19
20 export function updateProduct(uuid, updatedProduct) {
21   const index = products.findIndex((product) => product.uuid === uuid);
22   if (index !== -1) {
23     products[index] = updatedProduct;
24   } else {
25     throw new Error("Producto no encontrado");
26   }
27 }
28
29 export function deleteProduct(uuid) {
30   const index = products.findIndex((product) => product.uuid === uuid);
31   if (index !== -1) {
32     products.splice(index, 1);
33   } else {
34     throw new Error("Producto no encontrado");
35   }
36 }
37
38 export function findProduct(query) {
39   const [category, title] = query.split(":").map((str) => str.trim());
40   return products.filter((product) => {
41     const matchesCategory = category
42       ? product.category.includes(category)
43       : true;
44     const matchesTitle = title ? product.title.includes(title) : true;
45     return matchesCategory && matchesTitle;
46   });
47 }
48
49 export function productListToHTML(lista, htmlElement) {
50   const html = lista.map((product) => product.toHTML()).join("");
51   htmlElement.innerHTML = html;
52 }
```


shopping_cart.js:

Define la clase `ShoppingCart`, que gestiona los productos en el carrito de compras. Permite agregar productos al carrito, actualizar su cantidad, eliminar productos y calcular el total.

```
1  import { getProductById } from "../data_handler.js";
2
3  export class ShoppingCartException extends Error {
4      constructor(message) {
5          super(message);
6          this.name = "CartException";
7      }
8  }
9
10 export class ProductProxy {
11     constructor(productUuid, amount) {
12         this.productUuid = productUuid;
13         this.amount = amount;
14     }
15 }
```

```

1  export class ShoppingCart {
2      #cartItems = [];
3      #productList = [];
4
5      addItem(productUuid, amount) {
6          const existingItem = this.#cartItems.find(item => item.productUuid ===
productUuid);
7          if (existingItem) {
8              existingItem.amount += amount;
9          } else {
10             this.#cartItems.push(new ProductProxy(productUuid, amount));
11         }
12     }
13
14     updateItem(productUuid, newAmount) {
15         const item = this.#cartItems.find(item => item.productUuid === productU
uid);
16         if (!item)
17             throw new ShoppingCartException("Producto no encontrado en el carri
to.");
18
19         if (newAmount < 0)
20             throw new ShoppingCartException("La cantidad no puede ser negativ
a.");
21
22         if (newAmount === 0) {
23             this.removeItem(productUuid);
24         } else {
25             item.amount = newAmount;
26         }
27     }
28
29     removeItem(productUuid) {
30         this.#cartItems = this.#cartItems.filter(item => item.productUuid !== p
roductUuid);
31     }
32
33     calculateTotal() {
34         return this.#cartItems.reduce((total, item) => {
35             const product = getProductById(item.productUuid);
36             return total + (product ? product.pricePerUnit * item.amount : 0);
37         }, 0);
38     }
39
40     showCart() {
41         return this.#cartItems.map(item => {
42             const product = getProductById(item.productUuid);
43             return {
44                 nombre: product?.title || "Producto no encontrado",
45                 cantidad: item.amount,
46                 precioUnitario: product?.pricePerUnit || 0,
47                 total: product ? product.pricePerUnit * item.amount : 0
48             };
49         });
50     }
51 }
52

```

Funcionamiento del sistema

El sistema realiza las siguientes operaciones para gestionar los productos y el carrito de compras:

1. Creación de productos:

- Los productos se crean utilizando la clase `Product` con parámetros como título, descripción, imagen, unidad de medida, stock, precio por unidad y categoría.
- Cada producto recibe un UUID único, lo que asegura que cada producto sea identificable.

2. Gestión de productos:

- Los productos se almacenan en un array global llamado `products`.
- Se pueden agregar nuevos productos con `createProduct()`, actualizar con `updateProduct()` o eliminar con `deleteProduct()`.
- Se puede buscar un producto por categoría y título usando `findProduct()`.

3. Carrito de compras:

- El carrito de compras se maneja a través de la clase `ShoppingCart`.
- Los productos se agregan al carrito con `addItem()`, y la cantidad se puede actualizar con `updateItem()`.
- Los productos se pueden eliminar con `removeItem()`.
- El total de la compra se calcula utilizando `calculateTotal()`.

Conclusión

El sistema desarrollado para la gestión de productos y el carrito de compras ha sido exitoso en su implementación, cumpliendo con los objetivos establecidos. Durante esta práctica, se logró una comprensión más profunda de cómo estructurar un sistema de gestión de inventarios y cómo implementar interacciones con un carrito de compras utilizando JavaScript. Además, se aprendió a utilizar clases y objetos para organizar el código de manera más eficiente y legible.

Lo que se aprendió:

- **Manejo de excepciones:** Aprendimos a crear y manejar excepciones personalizadas, como `ProductException` y `ShoppingCartException`, lo cual es crucial para gestionar errores de manera controlada y evitar que el sistema falle.
- **Validación de datos:** Se comprendió la importancia de validar los datos antes de agregarlos al sistema. Esto garantiza la integridad de la información y evita errores que podrían afectar el funcionamiento del sistema.
- **Manipulación de arrays:** A través del uso de funciones como `createProduct()`, `updateProduct()`, y `deleteProduct()`, se trabajó con arrays para almacenar y manipular productos, lo cual es esencial para la gestión de inventarios.
- **Interacción con el carrito de compras:** Se comprendió cómo manejar un carrito de compras, permitiendo agregar, actualizar, eliminar productos y calcular el total, lo cual es una funcionalidad básica para cualquier sistema de e-commerce.

Tabla

Sección	Puntuación	
Clase Product	15 puntos	✓
Clase ShoppingCart	15 puntos	✓
Uso de excepciones y validaciones	10 puntos	✓
Funciones estáticas de Product	15 puntos	✓
CRUD del carrito de compras	20 puntos	✓
CRUD en <code>data_handler.js</code> y <code>productListToHTML</code>	30 puntos	✓
Pruebas y ejemplos	5 puntos	✓
Obtener el total del carrito de compras	5 puntos	✓

Pruebas

```
1 function createTestProducts() {
2   const product1 = new Product("Laptop", "Una laptop potente", "laptop.jpg", "pieza", 10, 1000, "Tecnologia");
3   createProduct(product1);
4   console.log("Producto creado:", product1);
5 }
6
7 function testCreateProduct() {
8   try {
9     const newProduct = new Product("Smartphone", "Smartphone de alta gama", "smartphone.jpg", "pieza", 20, 500, "Tecnologia");
10    createProduct(newProduct);
11    console.log("Producto creado:", newProduct);
12  } catch (error) {
13    console.error("Error al crear producto:", error.message);
14  }
15 }
16
17 function testUpdateProduct() {
18   const product = getProducts()[0];
19   if (product) {
20     const updatedProduct = new Product("Laptop", "Laptop actualizada", "laptop_actualizada.jpg", "pieza", 20, 900, "Tecnologia");
21     updateProduct(product.uuid, updatedProduct);
22     console.log("Producto actualizado:", updatedProduct);
23   }
24 }
25
26 function testShoppingCart() {
27   const cart = new ShoppingCart();
28   const product1 = getProducts()[0];
29
30   cart.addItem(product1.uuid, 2);
31   console.log("Carrito después de agregar producto:", cart.showCart());
32
33   const total = cart.calculateTotal();
34   console.log("Total del carrito:", total);
35 }
36
37 function runTests() {
38   console.clear();
39   createTestProducts();
40   testCreateProduct();
41   testUpdateProduct();
42   testShoppingCart();
43 }
44
45 runTests();
```

```
Producto creado: Product {
  _uuid: 'bdb8f1cb-b274-4aae-8cd4-d233e4edc7a7',
  _title: 'Laptop',
  _description: 'Una laptop potente',
  _imageUrl: 'laptop.jpg',
  _unit: 'pieza',
  _stock: 10,
  _pricePerUnit: 1000,
  _category: 'Tecnología'
}
Producto creado: Product {
  _uuid: 'bca9fbf9-d8eb-4b2b-9e95-0d8e1bef6548',
  _title: 'Smartphone',
  _description: 'Smartphone de alta gama',
  _imageUrl: 'smartphone.jpg',
  _unit: 'pieza',
  _stock: 20,
  _pricePerUnit: 500,
  _category: 'Tecnología'
}
Producto actualizado: Product {
  _uuid: '7e3fbef7-850f-4cd7-8e74-64c0d9aa0994',
  _title: 'Laptop',
  _description: 'Laptop actualizada',
  _imageUrl: 'laptop_actualizada.jpg',
  _unit: 'pieza',
  _stock: 20,
  _pricePerUnit: 900,
  _category: 'Tecnología'
}
Carrito después de agregar producto: [ { nombre: 'Laptop', cantidad: 2, precioUnitario: 900, total: 1800 } ]
Total del carrito: 1800

C:\Users\jorge\OneDrive\Documentos\Visual Studio\HTML\Temas de Clase\Mes 03\Practica 2>
PS C:\Users\jorge\OneDrive\Documentos\Visual Studio> |
```